

Emulador de uma ISA Mínima de Ponto Flutuante para Computação Numérica

Caio Vilor Brandão

Universidade Federal do ABC (UFABC)

Email: caio.brandao@aluno.ufabc.edu.br

Mateus Coelho Silva (Orientador)

Universidade Federal do ABC (UFABC)

Email: coelho.mateus@ufabc.edu.br

Resumo—Este artigo descreve o projeto e implementação de uma arquitetura mínima de ponto flutuante e de um emulador em software capazes de executar programas em uma ISA reduzida para computação numérica básica. Foram implementadas operações de $\sin(x)$, $\cos(x)$, $x^{1/n}$ e $\log_b(x)$ utilizando apenas um pequeno conjunto de instruções de carga, armazenamento, aritmética e desvio condicional, com um limite de doze instruções na ISA. Discutimos as decisões de projeto da arquitetura e da linguagem de montagem, a estrutura do emulador em Python, os métodos numéricos adotados e os desafios de mapeá-los para uma ISA tão restrita. Por fim, apresentamos resultados que evidenciam a correteza das implementações e discutimos implicações de desempenho e precisão.

Index Terms—ponto flutuante, ISA mínima, emulador, métodos numéricos, arquitetura de computadores.

I. INTRODUÇÃO

Arquiteturas didáticas minimalistas são uma ferramenta valiosa no ensino de organização e arquitetura de computadores. Ao reduzir o conjunto de instruções e a complexidade do hardware, torna-se mais fácil visualizar o caminho dos dados, o papel dos registradores, da memória, das instruções de desvio e a interação com algoritmos de mais alto nível.

Neste trabalho, foi projetada uma *Instruction Set Architecture* (ISA) mínima de ponto flutuante, acompanhada de um emulador em Python, com o objetivo de suportar a implementação de métodos numéricos básicos utilizando apenas um conjunto reduzido de instruções (no máximo doze). Foram escolhidos quatro problemas de referência:

- cálculo de $\sin(x)$ por série de Taylor;
- cálculo de $\cos(x)$ por série de Taylor;
- cálculo de $x^{1/n}$ por método de Newton;
- cálculo de $\log_b(x)$ por meio de aproximações de $\ln(x)$.

Esses métodos têm expressões matemáticas bem definidas, demandam laços, controle de fluxo condicional e uso intensivo de operações em ponto flutuante. Ao mesmo tempo, podem ser implementados sem instruções complexas, desde que o ISA ofereça um mínimo de operações aritméticas e de controle.

O objetivo principal não é desempenho absoluto, mas sim:

- 1) evidenciar como um conjunto pequeno de instruções pode ser suficiente para implementar algoritmos numéricos não triviais;
- 2) mostrar o impacto das restrições da ISA nas escolhas de implementação;
- 3) fornecer uma base para experimentos e extensões em trabalhos futuros.

II. ARQUITETURA E ISA

A. Organização de memória e registradores

A arquitetura proposta possui as seguintes características:

- **Registradores de propósito geral:** 8 registradores, denominados $R0$ a $R7$, todos representando valores em ponto flutuante.
- **Memória principal:** 64 posições de memória, endereçadas de $MEM[0]$ a $MEM[63]$, também em ponto flutuante.
- **Program Counter (PC):** registrador interno que aponta para o índice da instrução atual no programa carregado.
- **Flags de comparação:** três flags internas, não endereçáveis como registradores de dados:
 - LT (flag_lt): verdadeira quando o operando da esquerda é menor;
 - EQ (flag_eq): verdadeira quando há igualdade;
 - GT (flag_gt): verdadeira quando o operando da esquerda é maior.

Os oito registradores de propósito geral são os únicos visíveis e manipuláveis no nível da ISA pelo programador. As flags de comparação fazem parte do estado interno da CPU, sendo escritas pela instrução CMP e lidas pelas instruções de desvio condicional (BZ , BLT). Essa distinção reflete processadores reais, nos quais flags de status não são contabilizados como registradores de propósito geral.

B. Conjunto de instruções

A ISA foi limitada a no máximo 12 instruções. A Tabela I resume o conjunto final, com formato e semântica.

Tabela I
INSTRUÇÕES DA ISA MÍNIMA DE PONTO FLUTUANTE

Instrução	Semântica (informal)
SET Rd, imm	$Rd \leftarrow imm$
MOV Rd, Rs	$Rd \leftarrow Rs$
LOAD Rd, [a]	$Rd \leftarrow MEM[a]$
STORE Rs, [a]	$MEM[a] \leftarrow Rs$
ADD Rd, R1, R2	$Rd \leftarrow R1 + R2$
SUB Rd, R1, R2	$Rd \leftarrow R1 - R2$
MUL Rd, R1, R2	$Rd \leftarrow R1 \times R2$
DIV Rd, R1, R2	$Rd \leftarrow R1 / R2$
CMP R1, R2	Atualiza LT , EQ , GT
BZ label	Se EQ então $PC \leftarrow label$
BLT label	Se LT então $PC \leftarrow label$
JMP label	$PC \leftarrow label$

C. Justificativa das escolhas de design

As principais decisões de projeto foram:

- **Aritmética em ponto flutuante:** como o objetivo é implementar métodos numéricos, todos os registradores e a memória operam com valores em ponto flutuante, eliminando a necessidade de conversões explícitas na ISA.
- **Ausência de instruções inteiras:** não há operações específicas para inteiros; contadores de laço e índices são representados como floats (por exemplo, $k = 0.0, 1.0, 2.0$). Isso simplifica a ISA, ao custo de ligeira perda de naturalidade para laços, mas é suficiente para um emulador didático.
- **Separação entre registradores de dados e flags:** as flags *LT*, *EQ* e *GT* são internas, seguindo o modelo de arquiteturas como x86 e ARM. Isso permite implementar desvios condicionais com uma única instrução de comparação genérica.
- **Instruções simples e ortogonais:** não há modos de endereçamento complexos, nem operações com imediatos aritméticos além de SET. Todas as demais operações trabalham apenas entre registradores, o que reduz o espaço de casos a serem implementados no emulador.
- **Desvios por rótulos simbólicos:** no assembly, desvios são escritos com labels (*loop:*, *done:*). O emulador resolve esses labels em uma passada de montagem, facilitando a escrita de programas em assembly e a leitura para fins didáticos.

Esse conjunto mínimo mostrou-se suficiente para implementar os quatro métodos numéricos propostos, inclusive com laços e recursões de termos de série.

III. IMPLEMENTAÇÃO DO EMULADOR

O emulador foi implementado em Python, no arquivo *cpu.py*, com o objetivo de manter o código simples e legível.

A. Estrutura geral

A classe CPU encapsula o estado da máquina:

- vetor *reg[0..7]* de registradores de ponto flutuante;
- vetor *mem[0..63]* de memória em ponto flutuante;
- contador de programa *pc*;
- dicionário *labels* mapeando nomes de labels para índices de instruções;
- lista *program* contendo pares (*opcode*, *args*).

O método *load_program* recebe o código assembly como string, realiza duas passadas e popula *program* e *labels*.

B. Montagem em duas passadas

Na primeira passada, o emulador:

- 1) percorre linha a linha;
- 2) ignora comentários e linhas em branco;
- 3) quando encontra um label (linha terminada em ":"), associa o label ao índice lógico de instrução acumulado;

- 4) quando encontra uma instrução, incrementa o índice de instrução.

Na segunda passada, o código:

- 1) remove novamente comentários e labels;
- 2) para cada linha de instrução, extrai o opcode e a lista de argumentos;
- 3) acrescenta um par (*opcode*, *args*) em *program*.

Essa abordagem permite que instruções de desvio (JMP, BZ, BLT) usem labels simbólicos, que são resolvidos em índices inteiros apenas na execução.

C. Execução de instruções

O método *step()* executa uma única instrução:

- lê o par (*op*, *args*) em *program[pc]*;
- incrementa *pc*;
- de acordo com o opcode, atualiza registradores, memória ou flags;
- em caso de JMP, BZ ou BLT, altera *pc* para o índice do label correspondente.

A execução contínua é feita por *run()*, que simplesmente chama *step()* em laço até que *pc* saia do intervalo do programa.

Para fins de depuração e análise, a aplicação principal (*main.py*) imprime, ao final de cada execução, o conteúdo completo dos registradores e da memória, permitindo verificar como a ISA foi utilizada por cada programa numérico.

IV. MÉTODOS NUMÉRICOS

Esta seção descreve os algoritmos numéricos implementados, indicando brevemente de onde vêm e como funcionam, e discute os desafios de mapeá-los para a ISA restrita.

A. Série de Taylor para $\sin(x)$

O método utilizado para $\sin(x)$ vem diretamente da *série de Taylor* dessa função, obtida em Cálculo a partir das derivadas sucessivas de $\sin(x)$ avaliadas em $x = 0$. Para funções analíticas, a série de Taylor fornece uma aproximação polinomial local da função.

A expansão em torno de zero é:

$$\sin(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!}. \quad (1)$$

Na prática, truncamos a soma em um número finito de termos N e obtemos uma aproximação.

Para evitar o custo de recomputar potências e fatoriais a cada termo, utiliza-se uma *recorrência* entre termos sucessivos:

$$t_0 = x, \quad t_{k+1} = -t_k \cdot \frac{x^2}{(2k+2)(2k+3)}. \quad (2)$$

Ou seja, cada termo é obtido a partir do anterior multiplicando por x^2 e por um fator racional que incorpora o sinal alternado e o denominador do fatorial. No programa *sin.asm*, essa recorrência é implementada apenas com MUL, DIV, ADD, SUB e o controle de laço com CMP e BLT. Um registrador mantém o valor de x^2 , outro o termo atual t_k , outro o acumulador do resultado, e um registrador contador representa k em ponto flutuante.

B. Série de Taylor para $\cos(x)$

O método para $\cos(x)$ também vem da teoria de séries de Taylor: derivando repetidamente $\cos(x)$ e avaliando em $x = 0$, obtém-se uma série que aproxima a função em torno da origem.

A expansão é:

$$\cos(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k}}{(2k)!}, \quad (3)$$

que é novamente truncada em um número finito de termos N .

Usa-se uma recorrência semelhante à do seno:

$$t_0 = 1, \quad t_{k+1} = -t_k \cdot \frac{x^2}{(2k+1)(2k+2)}. \quad (4)$$

Assim, em vez de calcular diretamente x^{2k} e $(2k)!$, o programa atualiza o termo atual a partir do anterior. O programa `cos.asm` segue a mesma estrutura do `sin.asm`: pré-calcula x^2 , inicializa o termo inicial e acumula o resultado sobre N termos.

Uma limitação importante é a ausência de *redução de argumento*: em bibliotecas numéricas reais, valores grandes de x são trazidos para um intervalo menor (por exemplo, $[-\pi, \pi]$) antes de aplicar a série. Aqui, essa etapa foi omitida para manter a ISA simples; a consequência é que, para $|x|$ muito grande, são necessários mais termos para obter boa precisão.

C. Método de Newton para $x^{1/n}$

O cálculo de $x^{1/n}$ é baseado no *método de Newton-Raphson* para encontrar raízes de equações não lineares. Dado $f(y)$, o método constrói uma sequência de aproximações a partir da linearização local de f em torno de uma estimativa atual, conceito típico de Cálculo Numérico.

Define-se

$$f(y) = y^n - x. \quad (5)$$

Queremos um y tal que $f(y) = 0$, ou seja, $y^n = x$. O método de Newton define a iteração:

$$y_{k+1} = y_k - \frac{f(y_k)}{f'(y_k)}. \quad (6)$$

Como

$$f'(y) = ny^{n-1}, \quad (7)$$

obtém-se a forma clássica:

$$y_{k+1} = \frac{1}{n} \left((n-1) y_k + \frac{x}{y_k^{n-1}} \right). \quad (8)$$

Em palavras: a cada iteração, o método combina o valor atual y_k com a correção $\frac{x}{y_k^{n-1}}$, produzindo uma nova estimativa y_{k+1} que, sob condições razoáveis, converge rapidamente para $x^{1/n}$.

O programa `root.asm` implementa essa iteração em três níveis:

- 1) Laço externo sobre o número de iterações de Newton;
- 2) Laço interno para computar y_k^{n-1} por multiplicações sucessivas (não há instrução de potência);

- 3) Atualização de y usando as operações aritméticas básicas.

Os parâmetros de entrada são x , n e o número de iterações, todos armazenados em memória. O resultado aproximado é escrito em `MEM[10]`. A ausência de operações inteiras obriga o contador do laço interno a ser armazenado em um registrador de ponto flutuante e comparado via `CMP/BLT`.

D. Aproximação de $\log_b(x)$ via séries de $\ln$

O método para o logaritmo em base b combina uma identidade elementar de logaritmos com uma série clássica para $\ln$ deduzida em Cálculo a partir da expansão em série de funções como $\operatorname{artanh}(t)$.

Primeiro, usa-se a identidade:

$$\log_b(x) = \frac{\ln(x)}{\ln(b)}, \quad (9)$$

de propriedades básicas de logaritmos.

Para aproximar $\ln(z)$, utiliza-se a mudança de variável

$$t = \frac{z-1}{z+1}, \quad (10)$$

que leva à expansão

$$\ln(z) = 2 \sum_{k=0}^{\infty} t_k, \quad (11)$$

com

$$t_0 = t, \quad t_{k+1} = t_k \cdot t^2 \cdot \frac{2k+1}{2k+3}. \quad (12)$$

Essa série pode ser vista como derivada da expansão em série da função $\operatorname{artanh}(t)$, e é particularmente estável para z próximo de 1 (isto é, para t pequeno), o que motiva seu uso em implementações numéricas.

Na prática, o programa `log.asm` executa esse procedimento duas vezes: primeiro para $z = x$, depois para $z = b$, obtendo aproximações de $\ln(x)$ e $\ln(b)$ em `MEM[10]` e `MEM[11]`. Em seguida, calcula o quociente e armazena $\log_b(x)$ em `MEM[12]`.

Os principais desafios nessa implementação são:

- garantir que $x > 0$ e $b > 0, b \neq 1$ (hipótese assumida pelo programa chamador);
- controlar o número de termos da série (`MEM[2]`) para balancear precisão e custo de execução;
- implementar a recorrência apenas com as instruções da ISA, sem divisão inteira ou controle de fluxo mais sofisticado.

V. RESULTADOS E AVALIAÇÃO

A. Metodologia de avaliação

A avaliação foca em dois aspectos:

- **corretude**: comparar o resultado produzido pela ISA com a função equivalente da biblioteca `math` do Python;
- **custo computacional**: analisar a complexidade em função do número de termos/iterações e discutir o impacto da ISA restrita.

Para cada método numérico, foram escolhidos alguns valores de entrada representativos. O programa principal (`main.py`) lê os argumentos do usuário, configura a memória da CPU, executa o programa assembly e imprime o resultado aproximado e o valor de referência em Python.

B. Exemplos de corretude

A Tabela II ilustra resultados típicos para $\sin(x)$ e $\cos(x)$ com um número moderado de termos na série.

Tabela II
EXEMPLO DE APROXIMAÇÕES PARA $\sin(x)$ E $\cos(x)$

x	Função	ISA	Python
0.5	$\sin$	≈ 0.4794	0.4794
0.5	$\cos$	≈ 0.8776	0.8776
1.0	$\sin$	≈ 0.8415	0.8415
1.0	$\cos$	≈ 0.5403	0.5403

Com um número de termos suficientemente grande (por exemplo, $N = 10$ ou $N = 20$), a aproximação obtida pela ISA coincide com o valor da biblioteca padrão até várias casas decimais, o que é adequado para fins didáticos.

Para a raiz n -ésima, a Tabela III mostra um caso simples:

Tabela III
EXEMPLO DE APROXIMAÇÃO DE $x^{1/n}$

x	n	ISA	Python
16	2	≈ 4.0	4.0
27	3	≈ 3.0	3.0

Para o logaritmo, a Tabela IV apresenta exemplos para bases e argumentos simples:

Tabela IV
EXEMPLO DE APROXIMAÇÕES PARA $\log_b(x)$

x	b	ISA	Python
8	2	≈ 2.99	3.0
100	10	≈ 1.93	2.0
e	e	≈ 1.0	1.0

Nesses casos, com um número adequado de termos da série de $\ln$, o valor produzido pelo emulador coincide com o valor fornecido pela função `math.log` do Python dentro de erro numérico esperado.

C. Discussão de desempenho

Do ponto de vista assintótico, os custos são:

- $\sin(x)$ e $\cos(x)$: custo $O(N)$ em número de iterações, onde N é o número de termos da série. Cada iteração envolve um número constante de instruções aritméticas e de controle.
- $x^{1/n}$: custo $O(N_{\text{iter}} \cdot n)$, pois cada iteração de Newton contém um laço interno de tamanho $n - 1$ para computar y^{n-1} por multiplicações sucessivas.
- $\log_b(x)$: custo $O(N)$ para cada cálculo de $\ln$, sendo necessário executar o processo duas vezes (para x e para b).

A restrição da ISA impede o uso de instruções de potência ou funções transcendentais, o que força o uso de iteradores explícitos e aumenta o número de instruções executadas. Por outro lado, isso torna o caminho dos dados mais transparente, ajudando a entender como métodos numéricos podem ser decompostos em operações básicas.

D. Estado final da CPU como ferramenta de validação

Uma funcionalidade útil do emulador é a possibilidade de imprimir, ao término de cada execução, o conteúdo de todos os registradores e de todas as posições de memória. Esse *dump* do estado da CPU permite:

- inspecionar o valor dos termos intermediários (por exemplo, o último t_k da série de Taylor);
- verificar o valor do contador de laço ao sair das iterações;
- confirmar se o resultado está de fato sendo armazenado na posição de memória convencional (tipicamente `MEM[10]`).

Esse mecanismo facilitou a depuração das rotinas em assembly e pode ser aproveitado em atividades de laboratório para que estudantes explorem o comportamento da ISA.

VI. CONCLUSÃO

Este trabalho apresentou o projeto de uma ISA mínima de ponto flutuante e de um emulador em Python capazes de executar programas de computação numérica básicos. Com apenas oito registradores de propósito geral, 64 posições de memória em ponto flutuante e um conjunto de doze instruções simples, foi possível implementar quatro algoritmos relevantes: séries de Taylor para $\sin(x)$ e $\cos(x)$, método de Newton para $x^{1/n}$ e uma aproximação de $\log_b(x)$ via séries de $\ln$.

O exercício de projetar a ISA e de mapear métodos numéricos para essa arquitetura evidenciou que:

- um conjunto pequeno e ortogonal de instruções é suficiente para expressar algoritmos numéricos não triviais;
- a ausência de instruções complexas força um entendimento mais profundo das recorrências e das dependências de dados;
- o custo em número de instruções pode ser relativamente alto, mas aceitável para fins didáticos.

Como trabalhos futuros, é possível:

- estender a ISA com instruções adicionais (por exemplo, `NEG`, `ABS`, operações com imediatos);
- implementar rotinas de redução de argumento para melhorar a precisão de $\sin(x)$ e $\cos(x)$;
- introduzir um contador de instruções no emulador para quantificar empiricamente o custo de cada algoritmo;
- explorar a tradução dessa ISA mínima para uma arquitetura real, discutindo mapeamento e otimizações.

No contexto de ensino de arquitetura de computadores, o emulador e os programas apresentados oferecem uma base concreta para discutir o relacionamento entre software, ISA e algoritmos numéricos, com um nível de complexidade controlado e transparente.